

Week 2 · Day 9: High-Performance Inference Engines

vLLM Continuous Batching, PagedAttention, SGLang & Speculative Decoding

Goal: Serve 5-10x more tokens/sec than naive HuggingFace inference by understanding and operating the two leading production inference engines. Master the algorithmic breakthroughs behind vLLM (PagedAttention + continuous batching), explore SGLang's structured generation primitives, and cut TTFT with speculative decoding.

Advanced topic context: This is Week 2 material. You are expected to already have a FastAPI server running (Day 5), understand quantization (Day 4), and have benchmarked CPU vs GPU throughput (Day 3). The gap between 'HuggingFace works' and 'production can handle 100 concurrent users' is almost entirely explained by this day.

PART A — Why Naive HuggingFace Inference Fails at Scale

The Three Fundamental Problems

When you wrap a HuggingFace model in FastAPI and serve real traffic, you immediately hit three hard walls that make naive serving completely unacceptable in production:

Problem	Root Cause	Impact
1. Static batching	Requests must be grouped before generation starts. All sequences wait for the slowest request to finish before any output is produced.	GPU utilization is low. Throughput is severely limited by long-tail requests.
2. KV cache fragmentation	OS-style memory allocation reserves a contiguous block for each sequence. As batch size increases, memory is wasted on fragmentation.	Batch size is capped by memory, not by GPU capacity.
3. Autoregressive serialisation	Each token requires a full forward pass. Without high-performance optimizations, requests are served sequentially.	Throughput is low. Parallelism benefits are not realized.

Quantifying the Gap

Rough throughput comparison on a single A100 80GB, LLaMA-3 8B FP16

Naive HuggingFace (synchronous, batch_size=1):
~45 tok/s generation speed
~5 concurrent users before latency degrades
~225 tok/s total system throughput (45 x 5, but actually sequential)

vLLM with continuous batching + PagedAttention:
~2,000-4,000 tok/s total system throughput
~100+ concurrent users with stable latency
~10-20x improvement in throughput

SGLang (optimized for structured outputs):
~3,000-5,000 tok/s on structured generation workloads
Superior for constrained decoding (JSON, regex)

```
# The gap is not hardware -- it is the serving algorithm.  
# vLLM and SGLang extract 80%+ GPU utilisation vs 20-30% for naive HF.
```

PART B — vLLM: PagedAttention & Continuous Batching

Innovation 1: PagedAttention

PagedAttention (introduced in the vLLM paper, 2023) applies the virtual memory / paging concept from OS design to the KV cache. Instead of allocating one large contiguous memory block per sequence (which fragments VRAM), PagedAttention divides the KV cache into fixed-size **blocks** (pages) of e.g. 16 tokens each. A **block table** maps each sequence's logical block indices to physical GPU memory blocks, just like a CPU page table maps virtual to physical addresses.

PagedAttention Memory Layout

```
# Traditional KV cache (contiguous allocation per sequence):
#
# Seq A [#####] ] <- allocated 2048 tokens, uses 412 -> waste!
# Seq B [#####] ] <- allocated 2048 tokens, uses 201 -> waste!
# Seq C [fragmentation gap] ] <- can't fit new sequence
#
# PagedAttention (paged allocation):
#
# Physical blocks: [B0][B1][B2][B3][B4][B5][B6][B7][B8][B9]...
#                  ^               ^
# Block table A: { 0->B0, 1->B3, 2->B7 } # non-contiguous, exact fit
# Block table B: { 0->B1, 1->B4 }
# Block table C: { 0->B2, 1->B5, 2->B8 } # allocated as needed, no waste
#
# Key properties:
# 1. Internal fragmentation reduced to at most 1 block (16 tokens) per sequence
# 2. Blocks can be SHARED between sequences (for prefix/prompt caching)
# 3. Block eviction policy (like OS page eviction) handles memory pressure
# 4. New sequences can be admitted as long as any free blocks exist
#
# Block size is a tunable parameter:
# --block-size 8 -> lower memory waste, higher block table overhead
# --block-size 16 -> default, good balance
# --block-size 32 -> lower overhead, slightly more waste
```

Copy-on-Write for Parallel Sampling

PagedAttention enables an important optimisation for beam search and parallel sampling: **copy-on-write (CoW) block sharing**. When sampling N completions from the same prompt, all N sequences share the same physical KV blocks for the prompt prefix. A block is only physically copied when a sequence diverges (writes a different token). This can reduce KV cache memory for beam search by up to N-1 times the prompt length.

Innovation 2: Continuous Batching

In static batching all requests in a batch must finish before the next batch starts. A single long request (1000 tokens) blocks 7 short requests (100 tokens each) from getting a GPU slot. Continuous batching (also called iteration-level scheduling) solves this by making scheduling decisions at the **token step level** rather than the request level. After every forward pass, vLLM's scheduler can:

1. **Remove** completed sequences from the batch immediately
2. **Add** new waiting sequences into the freed slots for the next step
3. **Preempt** (swap to CPU) sequences that need too many KV blocks
4. **Resume** preempted sequences when blocks become available

This keeps the GPU batch full at all times and prevents head-of-line blocking. The throughput improvement is typically **5-23x over static batching** for real-world mixed-length request distributions.

vLLM Architecture Overview

Component	Role	Key Config
LLMEngine	Core engine: scheduler + worker coordination	Central orchestrator
Scheduler	Continuous batching decisions per token step	max_num_seqs, max_num_batched_tokens
BlockManager	PagedAttention physical/logical block mapping	--block-size, --gpu-memory-utilization
Worker (GPU)	Executes model forward pass per step	--tensor-parallel-size
KVCacheManager	Allocates/frees/swaps blocks across GPU/CPU	--swap-space (CPU swap in GB)
AsyncLLMEngine	Async wrapper for concurrent request handling	Default for vllm serve
OpenAI API Server	HTTP layer: /v1/chat/completions, /v1/completions	--port, --host

Practical Implementation — vLLM

B.1 Installation

```
# vLLM requires CUDA 11.8+ and Python 3.9+
pip install vllm

# Verify GPU is detected
python -c "import vllm; print(vllm.__version__)"
python -c "import torch; print(torch.cuda.get_device_name(0))"

# For specific CUDA versions (if pip install fails):
# pip install vllm --extra-index-url https://download.pytorch.org/whl/cu121
```

B.2 Serve a Model with vLLM (CLI)

```
# Serve LLaMA-3 8B with default settings
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
    --port 8000 \
    --dtype float16

# Key tuning flags explained:
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
    --port 8000 \
    --dtype float16 \
    --max-model-len 8192 \           # context window (reduce to save KV cache VRAM)
    --gpu-memory-utilization 0.90 \ # fraction of GPU VRAM for KV cache (default 0.90)
    --max-num-seqs 256 \           # max concurrent sequences in the batch
```

```

--max-num-batched-tokens 4096 \ # max tokens processed per forward pass
--block-size 16 \                # KV cache page size in tokens
--swap-space 4 \                 # GB of CPU RAM for preempted sequence KV swap
--served-model-name llama3       # name exposed via /v1/models

# AWQ quantised model (fits on 8GB GPU)
vllm serve TheBloke/Llama-2-7B-Chat-AWQ \
  --quantization awq \
  --dtype float16 \
  --max-model-len 4096

# Verify server is up
curl http://localhost:8000/health
curl http://localhost:8000/v1/models | python -m json.tool

```

B.3 Python API (Offline Batching)

```

from vllm import LLM, SamplingParams
import time

# LLM() loads the model for offline (non-server) batch inference
llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    dtype="float16",
    gpu_memory_utilization=0.85,
    max_model_len=4096,
)

sampling_params = SamplingParams(
    temperature=0.7,
    top_p=0.9,
    max_tokens=200,
    stop=["</s>", "<|eot_id|>"], # model-specific EOS tokens
)

prompts = [
    "Explain PagedAttention in one paragraph.",
    "What is continuous batching and why does it matter?",
    "Compare vLLM and plain HuggingFace for production serving.",
    "What are the three pillars of production LLM infrastructure?",
    "Describe the KV cache block table data structure.",
]

t0 = time.perf_counter()
outputs = llm.generate(prompts, sampling_params) # batches all prompts together
elapsed = time.perf_counter() - t0

total_out_tokens = sum(len(o.outputs[0].token_ids) for o in outputs)
total_in_tokens = sum(len(o.prompt_token_ids) for o in outputs)

print(f"Processed {len(prompts)} prompts simultaneously")
print(f"Total input tokens : {total_in_tokens}")
print(f"Total output tokens : {total_out_tokens}")
print(f"Wall time : {elapsed:.2f}s")
print(f"System throughput : {total_out_tokens / elapsed:.0f} tok/s")

for i, out in enumerate(outputs):

```

```

print(f"\n--- Prompt {i+1} ---")
print(f"  Input  : {out.prompt[:60]}...")
print(f"  Output : {out.outputs[0].text[:120]}...")
print(f"  Tokens : {len(out.outputs[0].token_ids)}")
print(f"  Reason : {out.outputs[0].finish_reason}")

```

B.4 OpenAI-Compatible Client against vLLM Server

```

from openai import OpenAI
import time, asyncio, httpx

client = OpenAI(base_url="http://localhost:8000/v1", api_key="vllm")

# Single request
t0 = time.perf_counter()
resp = client.chat.completions.create(
    model="llama3",
    messages=[
        {"role": "system", "content": "You are a concise technical assistant."},
        {"role": "user", "content": "What is PagedAttention?"},
    ],
    max_tokens=150,
    temperature=0.7,
)
elapsed = time.perf_counter() - t0

print(resp.choices[0].message.content)
print(f"\nPrompt tokens      : {resp.usage.prompt_tokens}")
print(f"Completion tokens : {resp.usage.completion_tokens}")
print(f"Wall time           : {elapsed:.3f}s")
print(f"Tok/s (generation): {resp.usage.completion_tokens / elapsed:.1f}")

# Concurrent requests -- this is where vLLM shines
async def concurrent_requests(n: int = 10):
    async_client = httpx.AsyncClient(timeout=120)
    prompts = [
        f"Explain concept {i}: {'KV cache' if i%3==0 else 'batching' if i%3==1 else 'quantization'}."
        for i in range(n)
    ]

    async def one_request(prompt, idx):
        t0 = time.perf_counter()
        r = await async_client.post(
            "http://localhost:8000/v1/chat/completions",
            json={"model": "llama3",
                  "messages": [{"role": "user", "content": prompt}],
                  "max_tokens": 100},
        )
        lat = time.perf_counter() - t0
        data = r.json()
        toks = data["usage"]["completion_tokens"]
        return idx, lat, toks

    t_start = time.perf_counter()
    results = await asyncio.gather(*[one_request(p, i) for i, p in enumerate(prompts)])
    wall = time.perf_counter() - t_start

```



```
sp = SamplingParams(max_tokens=N_TOKENS, temperature=0, ignore_eos=False)
```


PART C — SGLang: Structured Generation Language

What is SGLang?

SGLang (Structured Generation Language) is an inference framework developed at UC Berkeley that goes beyond raw throughput to address a different bottleneck: **the cost of complex multi-step LLM programs**. Real production workloads rarely consist of single-turn completions. They involve chains of LLM calls, constrained outputs (JSON, regex), parallel branches, and shared prefixes. SGLang provides a Python DSL to express these patterns and optimises their execution through RadixAttention (prefix caching) and constrained decoding.

SGLang vs vLLM — When to Use Which

Dimension	vLLM	SGLang
Primary strength	Maximum raw throughput, standard serving	Complex multi-step programs, structured output
Prefix caching	Automatic (prefix cache flag)	RadixAttention -- radix tree, more efficient
Constrained decoding	Via outlines / lm-format-enforcer	Native -- grammar, regex, JSON schema built-in
Multi-step programs	Manual: chain API calls yourself	Native: fork(), join(), gen() DSL
Parallel sampling	Via n parameter in SamplingParams	Native fork() with shared prefix blocks
OpenAI compatibility	Full /v1/* endpoints	Full /v1/* endpoints
Production maturity	Very high -- widely deployed	High -- fast-growing adoption
Best use case	Chat, general serving, high concurrency	Agents, RAG, JSON extraction, structured gen

RadixAttention -- SGLang's Prefix Cache

SGLang's RadixAttention stores KV cache entries in a **radix tree** (a compressed trie) indexed by token sequence. When a new request shares a prefix with any cached sequence, the shared KV blocks are reused exactly -- no recomputation. This is more memory-efficient than vLLM's prefix cache because the radix tree can share arbitrary-length common prefixes across many requests, not just fixed system prompts. For agent workloads where many requests share a long tool schema or document context, this can reduce TTFT by 60-80%.

Practical Implementation -- SGLang

C.1 Installation

```
# SGLang requires CUDA 11.8+, Python 3.9+
pip install "sglang[all]"

# Or for a specific CUDA version:
pip install "sglang[all]" --find-links https://flashinfer.ai/whl/cu121/torch2.3/

# Verify install
```

```
python -c "import sglang; print(sglang.__version__)"
```

C.2 Launch the SGLang Server

```
# Launch as a background inference server (OpenAI-compatible API)
python -m sglang.launch_server \
    --model-path meta-llama/Meta-Llama-3-8B-Instruct \
    --port 30000 \
    --dtype float16 \
    --mem-fraction-static 0.85 \      # fraction of VRAM for static KV cache
    --context-length 8192 \
    --tp 1                          # tensor parallelism degree

# Alternatively -- programmatic launch inside Python scripts
import sglang as sgl

runtime = sgl.Runtime(
    model_path="meta-llama/Meta-Llama-3-8B-Instruct",
    port=30000,
    dtype="float16",
)
sgl.set_default_backend(runtime)
```

C.3 SGLang DSL -- Core Primitives

[illegible]


```

@sgl.function
def batch_extract(s, texts):
    # Run the same program on a batch of inputs in parallel
    states = simple_qa.run_batch(
        [{"question": t} for t in texts],
        num_threads=8,
        progress_bar=True,
    )
    return states

```

C.5 Constrained Decoding -- JSON Schema

```

import sglang as sgl
import json

# SGLang can constrain output to a JSON schema using a grammar-based decoder.
# This guarantees valid JSON -- no post-processing needed.

SCHEMA = {
    "type": "object",
    "properties": {
        "model_name": {"type": "string"},
        "params_B": {"type": "number"},
        "best_quant": {"type": "string", "enum": ["FP16", "AWQ", "GPTQ", "GGUF"]},
        "vram_needed_gb": {"type": "number"},
        "recommended_gpu": {"type": "string"},
    },
    "required": ["model_name", "params_B", "best_quant", "vram_needed_gb", "recommended_gpu"]
}

@sgl.function
def structured_model_info(s, model_description):
    s += sgl.system(
        "You are an AI infrastructure expert. "
        "Return a JSON object with model deployment recommendations."
    )
    s += sgl.user(f"Provide deployment info for: {model_description}")
    s += sgl.assistant(
        sgl.gen(
            "info",
            max_new_tokens=200,
            temperature=0,
            json_schema=json.dumps(SCHEMA),    # enforce JSON schema at decode time
        )
    )

state = structured_model_info.run(
    model_description="LLaMA 3 8B for production chat serving on a single GPU"
)
result = json.loads(state["info"])
print(json.dumps(result, indent=2))
# Guaranteed to parse -- no try/except needed

```

C.6 Benchmark SGLang vs vLLM

```

import time, requests, json

```

```

PROMPTS = [
    "What is continuous batching in LLM serving?",
    "Explain PagedAttention in two sentences.",
    "When would you use SGLang over vLLM?",
    "What is speculative decoding?",
    "Describe the KV cache eviction policy in vLLM.",
] * 4    # 20 prompts

def bench_server(base_url: str, model: str, label: str):
    import asyncio, httpx

    async def run_all():
        async with httpx.AsyncClient(timeout=300) as client:
            tasks = []
            for p in PROMPTS:
                tasks.append(client.post(
                    f"{base_url}/v1/chat/completions",
                    json={"model": model,
                        "messages": [{"role": "user", "content": p}],
                        "max_tokens": 80, "temperature": 0},
                ))
            t0 = time.perf_counter()
            resps = await asyncio.gather(*tasks)
            elapsed = time.perf_counter() - t0

            total_out = sum(
                r.json()["usage"]["completion_tokens"]
                for r in resps if r.status_code == 200
            )
            return elapsed, total_out

    elapsed, total_toks = asyncio.run(run_all())
    tps = total_toks / elapsed
    print(f"{label:<25} {len(PROMPTS):>8} prompts "
          f"{total_toks:>8} toks {elapsed:>7.1f}s {tps:>8.0f} tok/s")

print(f"'Engine':<25} {'Prompts':>8} {'Toks':>8} {'Time':>7} {'Tok/s':>9}")
print("-" * 68)
bench_server("http://localhost:8000", "llama3", "vLLM")
bench_server("http://localhost:30000", "meta-llama/Meta-Llama-3-8B-Instruct", "SGLang")

```

PART D — Speculative Decoding

The Problem: TTFT is Dominated by Draft Latency

In autoregressive generation, each token requires one full forward pass through the model. A 70B parameter model might take 50ms per token, giving 20 tok/s. Speculative decoding exploits the fact that **verifying N tokens is no more expensive than generating 1 token** in a single forward pass, because attention can process all N tokens in parallel during verification.

How Speculative Decoding Works

```
# Algorithm overview:
#
# 1. DRAFT PHASE: A small, fast "draft model" (e.g. 1B params) autoregressively
#    generates K candidate tokens quickly (e.g. K=5 tokens).
#
# 2. VERIFY PHASE: The large "target model" (e.g. 70B) runs ONE forward pass
#    to compute the probability of all K+1 positions simultaneously.
#
# 3. ACCEPTANCE: Compare draft token probabilities to target probabilities.
#    - If draft token is "good enough" (acceptance criterion met): accept it.
#    - If draft token is rejected: resample from the target distribution and stop.
#
# 4. RESULT: Between 1 and K+1 tokens are produced per draft+verify cycle.
#    The target model's output distribution is preserved exactly (no quality loss).
#
# Key insight:
#   Standard decode:      N tokens = N target model forward passes
#   Speculative decode:   N tokens ~ N/acceptance_rate verify passes
#                           (with acceptance_rate typically 0.6-0.9)
#
# Speedup is highest when:
#   - Draft model is fast (1B-3B params)
#   - Acceptance rate is high (predictable text, code)
#   - Target model is slow (70B+ params)
#   - Batch size is 1 (the verify overhead doesn't amortize over many requests)
#
# Theoretical speedup formula:
#   speedup = (1 + K * acceptance_rate) / (1 + K * draft_cost_ratio)
#
# Where:
#   K           = draft tokens per cycle (e.g. 5)
#   acceptance_rate = fraction of draft tokens accepted (e.g. 0.75)
#   draft_cost_ratio = draft_model_time / target_model_time (e.g. 0.1 for 1B vs 70B)
#
# Example: K=5, acceptance=0.75, draft_cost=0.1
#   speedup = (1 + 5*0.75) / (1 + 5*0.1) = 4.75 / 1.5 = 3.2x TTFT improvement
```

D.1 Enable Speculative Decoding in vLLM

```
# Method 1: CLI -- use a dedicated draft model
vllm serve meta-llama/Meta-Llama-3-70B-Instruct \
  --speculative-model meta-llama/Meta-Llama-3-8B-Instruct \
  --num-speculative-tokens 5 \                # K: draft tokens per cycle
```

```

--speculative-draft-tensor-parallel-size 1 \
--dtype float16 \
--tensor-parallel-size 4                # target model needs 4 GPUs for 70B

# Method 2: n-gram draft (no separate model needed -- uses prompt patterns)
# Excellent for document processing, summarization, code generation
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
--speculative-model "[ngram]" \
--num-speculative-tokens 5 \
--ngram-prompt-lookup-max 4 \          # max n-gram window size to match
--ngram-prompt-lookup-min 1 \
--dtype float16

# Verify speculative decoding is active
curl http://localhost:8000/v1/models | python -m json.tool

```

D.2 Python API with Speculative Decoding

```

from vllm import LLM, SamplingParams
import time

# LLM with speculative decoding (draft + target model)
llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",    # target model
    speculative_model="Qwen/Qwen2-0.5B-Instruct",    # draft model (small + fast)
    num_speculative_tokens=5,                        # K: drafts per cycle
    dtype="float16",
    gpu_memory_utilization=0.85,
)

# N-gram speculative decoding (no separate draft model)
llm_ngram = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    speculative_model="[ngram]",
    num_speculative_tokens=5,
    ngram_prompt_lookup_max=4,
    dtype="float16",
)

sp = SamplingParams(max_tokens=200, temperature=0.0) # temp=0 maximises acceptance rate

# Benchmark standard vs speculative
def benchmark(engine, label, prompts):
    t0 = time.perf_counter()
    outs = engine.generate(prompts, sp)
    elapsed = time.perf_counter() - t0
    total_toks = sum(len(o.outputs[0].token_ids) for o in outs)
    print(f"{label:<35} {total_toks:>8} toks {elapsed:>7.2f}s {total_toks/elapsed:>8.0f} tok/s")

PROMPTS = ["Summarise the history of neural networks in detail."] * 8

print(f"{'Engine':<35} {'Toks':>8}          {'Time':>7} {'Tok/s':>9}")
print("-" * 65)

llm_standard = LLM("meta-llama/Meta-Llama-3-8B-Instruct", dtype="float16")
benchmark(llm_standard, "Standard (no speculation)", PROMPTS)

```

```
benchmark(llm_ngram, "N-gram speculative (K=5)", PROMPTS)
benchmark(llm, "Draft model speculative (K=5)", PROMPTS)
```

D.3 Measuring Acceptance Rate

```
# vLLM exposes speculative decoding metrics via Prometheus (/metrics)
# Key metrics to monitor:

# vllm:spec_decode_draft_acceptance_rate
#   - Fraction of draft tokens accepted by target model
#   - Target: > 0.7 for meaningful speedup
#   - Low acceptance (<0.4): draft model is poorly matched, consider different draft

# vllm:spec_decode_efficiency
#   - Actual speedup achieved (actual_tokens / standard_tokens_per_wall_time)
#   - Target: > 1.5x for speculation to be worthwhile

# Acceptance rate by task type (empirical observations):
ACCEPTANCE_RATES = {
    "Code completion (Python)": "0.80 - 0.92",
    "Repetitive document text": "0.75 - 0.88",
    "Factual Q&A": "0.65 - 0.80",
    "Open-ended chat": "0.55 - 0.75",
    "Creative writing": "0.40 - 0.60",
    "JSON extraction": "0.70 - 0.85",
}

print("Typical acceptance rates by workload:")
for task, rate in ACCEPTANCE_RATES.items():
    print(f" {task:<35}: {rate}")

# Scrape live acceptance rate from vLLM metrics endpoint
import requests, re

metrics_text = requests.get("http://localhost:8000/metrics").text
for line in metrics_text.split("\n"):
    if "spec_decode" in line and not line.startswith("#"):
        print(line)
```


Testing & Metrics

Comprehensive Throughput Benchmark

```
# Use vLLM's built-in benchmark script for rigorous throughput measurement
# This is the standard benchmarking tool used in the vLLM paper

# Download benchmark script
wget https://raw.githubusercontent.com/vllm-project/vllm/main/benchmarks/benchmark_serving.py

# Run against a live vLLM server
python benchmark_serving.py \
  --backend vllm \
  --host localhost \
  --port 8000 \
  --model meta-llama/Meta-Llama-3-8B-Instruct \
  --dataset-name sharegpt \
  --dataset-path ShareGPT_V3_unfiltered_cleaned_split.json \
  --num-prompts 200 \
  --request-rate 10 \           # requests per second (inf = all at once)
  --percentile-metrics p50,p95,p99

# Key output metrics to record:
# Throughput           : X requests/s, Y tokens/s
# TTFT mean/p50/p95/p99 : time to first token
# TPOT mean/p50/p95     : time per output token (TBT)
# ITL (inter-token latency): same as TPOT
# E2E latency           : total request time

# Also run offline batch benchmark (measures peak throughput without network)
python benchmark_throughput.py \
  --model meta-llama/Meta-Llama-3-8B-Instruct \
  --input-len 512 \
  --output-len 128 \
  --num-prompts 100 \
  --dtype float16
```

Key Metrics and Targets

Metric	Formula / Source	Production Target (8B, A100)
System throughput (tok/s)	Total output tokens / wall time	>1,500 tok/s (vLLM, bs=32)
TTFT p50	vLLM benchmark_serving.py	< 500ms (GPU, 512 tok prompt)
TTFT p99	vLLM benchmark_serving.py	< 2s (GPU)
TPOT (TBT) p50	Time between tokens	< 30ms (>33 tok/s)
GPU utilisation	nvidia-smi --query-gpu=utilization.gpu	> 80% under load
KV cache usage	vllm:gpu_cache_usage_perc (Prometheus)	60-85% (headroom for spikes)
Acceptance rate (spec)	vllm:spec_decode_draft_acceptance_rate	> 0.70 for net speedup
HF vs vLLM speedup	vllm_tps / hf_tps	5-15x on real workloads

Essential Shell Commands

```
# Monitor GPU utilisation and memory in real time
watch -n 1 nvidia-smi

# Monitor vLLM Prometheus metrics
curl -s http://localhost:8000/metrics | grep -E "vllm|gpu_cache|num_requests"

# Check current batch size and queue depth
curl -s http://localhost:8000/metrics | grep "num_requests"
# vllm:num_requests_running -- sequences currently in the batch
# vllm:num_requests_waiting -- sequences queued (waiting for KV blocks)
# vllm:num_requests_swapped -- sequences swapped to CPU

# Check KV cache utilisation
curl -s http://localhost:8000/metrics | grep "cache_usage"
# vllm:gpu_cache_usage_perc -- fraction of KV cache blocks in use (0.0 - 1.0)
# vllm:cpu_cache_usage_perc -- CPU swap utilisation

# Inspect block manager state (debug mode -- adds overhead)
VLLM_LOGGING_LEVEL=DEBUG vllm serve ... 2>&1 | grep "block"

# Quick smoke test
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"llama3", "messages":[{"role":"user", "content":"hi"}], "max_tokens":5}'
```

Tuning Guide -- Common Knobs

Flag	Default	Tuning Guidance
--gpu-memory-utilization	0.90	Lower (0.80) if OOM; raise (0.95) for throughput
--max-num-seqs	256	Lower if KV cache OOM; raise for throughput
--max-num-batched-tokens	auto	Set to context_len x max_seqs for max batch
--block-size	16	Keep at 16; raise to 32 if block table overhead visible
--swap-space	4 GB	Raise if many requests get preempted (metrics show swapped > 0)
--num-speculative-tokens	N/A	Start at 5; raise if acceptance > 0.8, lower if < 0.6
--max-model-len	auto	Reduce to free KV cache VRAM for higher concurrency
--tensor-parallel-size	1	Match to number of GPUs; must divide num_attention_heads

Key Takeaways

- **Naive HF inference wastes 70-80% of GPU capacity** due to static batching and contiguous KV allocation. vLLM fixes both with continuous batching + PagedAttention.
- **PagedAttention** is the OS virtual memory concept applied to KV cache: fixed-size blocks, a block table, copy-on-write sharing, and eviction under pressure. It eliminates KV cache fragmentation and enables near-zero waste allocation.

- **Continuous batching** schedules at the token step level -- finished sequences leave and new ones join the batch after every forward pass, keeping GPU utilisation above 80% under real mixed-length workloads.
- **SGLang** adds a structured generation DSL on top of high-throughput serving. Use it when your workload involves chains of LLM calls, parallel branches (fork/join), or guaranteed-valid JSON/regex outputs. Use vLLM for standard chat/completion serving.
- **Speculative decoding** cuts TTFT by running a cheap draft model to propose K tokens per cycle and verifying all K in one target model forward pass. Speedup is 1.5-3x on predictable text (code, documents); lower on creative generation. Use n-gram drafting as a zero-cost starting point before investing in a separate draft model.
- **Tune --gpu-memory-utilization first**: it is the single knob with the largest impact on both maximum batch size and KV cache capacity.

Next topic: **Day 10 -- Attention Optimizations**: Flash Attention IO-aware tiling, PagedKV internals, and advanced KV cache tuning in vLLM (prefix caching, --gpu-memory-utilization, cache hit rate measurement).